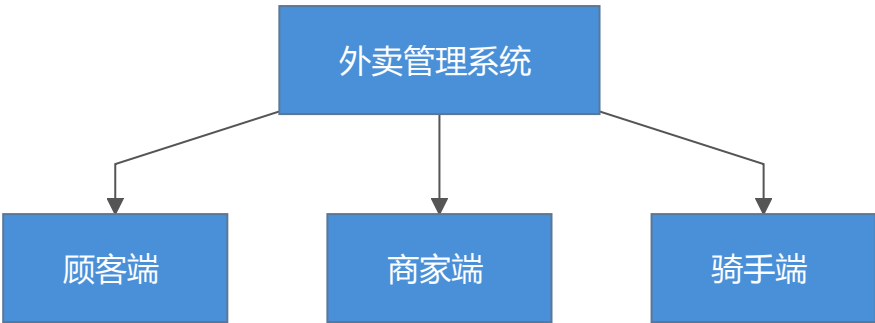


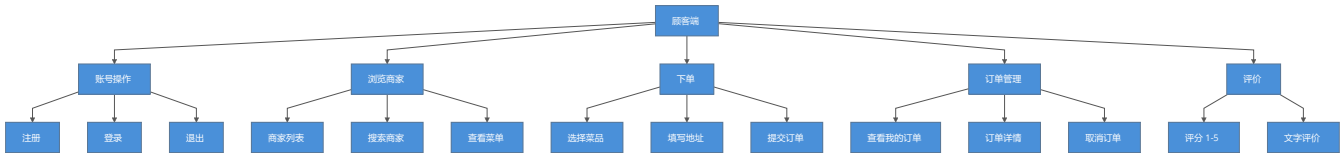
外卖管理系统 — 系统设计文档

一、功能结构图

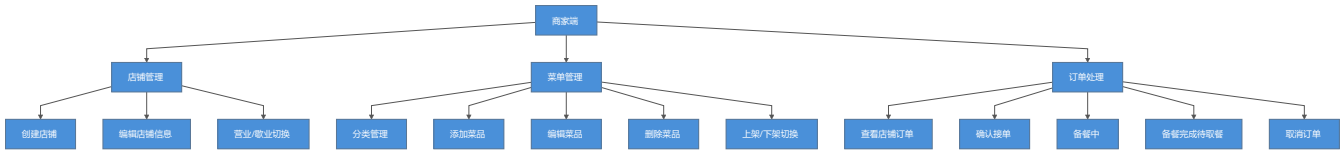
总览



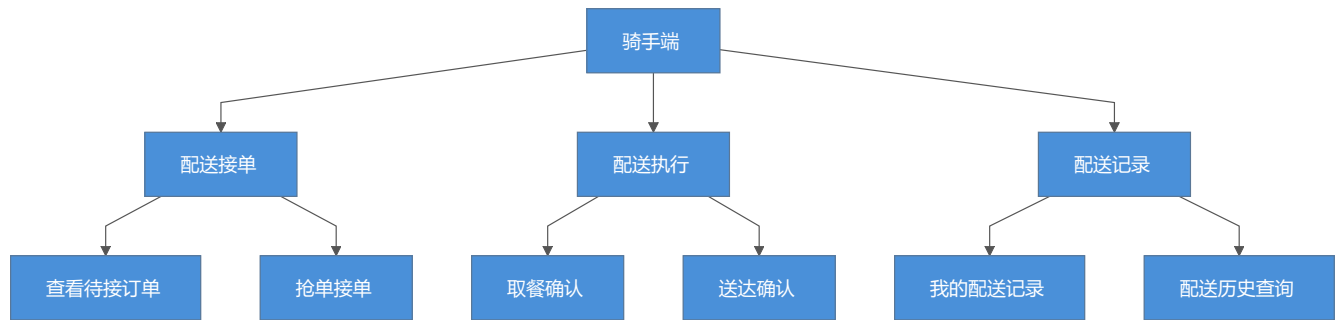
1.1 顾客端功能



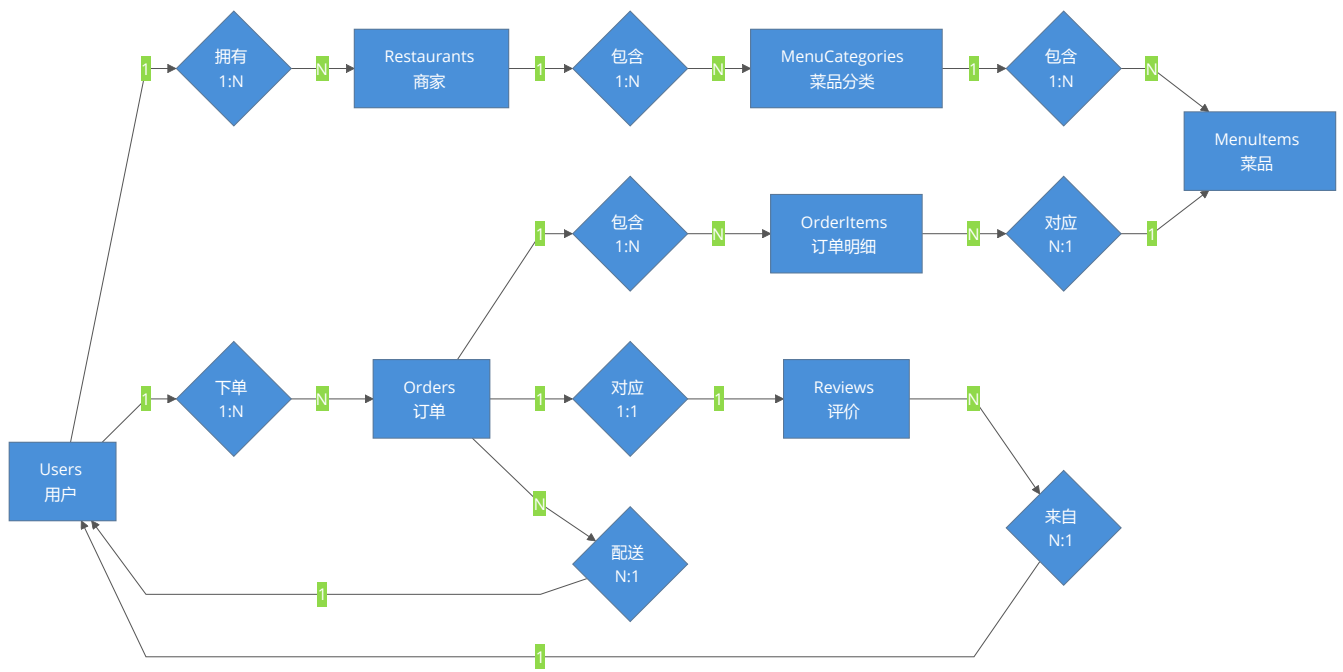
1.2 商家端功能



1.3 骑手端功能



二、E-R 图 (实体关系图)



三、订单状态流转

订单共有 8 种状态，分两个阶段流转：

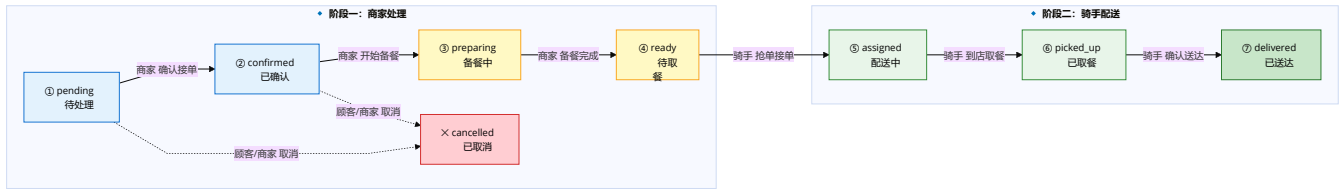
- 阶段一（商家处理）：pending → confirmed → preparing → ready
- 阶段二（骑手配送）：assigned → picked_up → delivered

ready 是两个阶段的交接点：商家备好餐后等待骑手取餐，骑手从 ready 状态介入。

cancelled（已取消）和 delivered（已送达）为终态，不可再向后流转。

备餐后不可取消：进入 preparing 后，订单不能再取消 —— 防止食物浪费。

状态流转图（按角色分阶段）



图例：● 蓝色 = 待确认/已确认 ● 黄色 = 备餐中/待取餐 ● 绿色 = 配送中/已取餐/已送达 ● 红色 = 已取消
实线箭头 = 正常流转 虚线箭头 = 取消（旁路）

状态说明

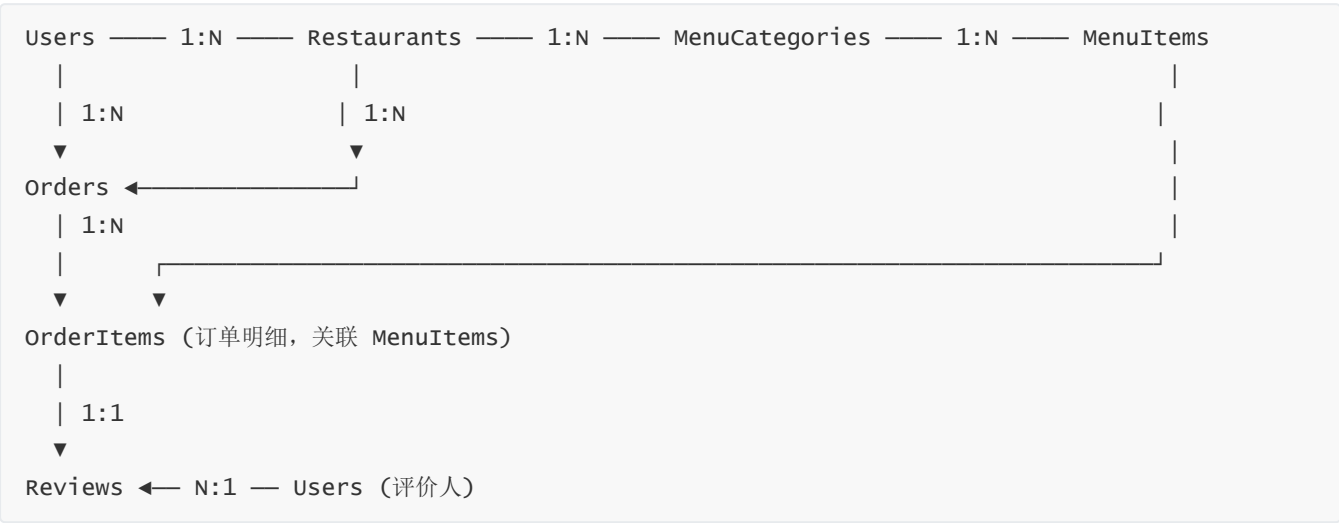
状态	中文名	所处阶段	说明
pending	待处理	阶段一	顾客已提交订单，等待商家确认

状态	中文名	所处阶段	说明
confirmed	已确认	阶段一	商家已确认接单，即将备餐
preparing	备餐中	阶段一	商家正在制作餐品
ready	待取餐	交接点	餐品备好，等待骑手取餐 ← 两阶段在此衔接
assigned	配送中	阶段二	骑手已接单，前往商家
picked_up	已取餐	阶段二	骑手已取餐，配送途中
delivered	已送达	终态	餐品送达，订单完成 ✓
cancelled	已取消	终态	订单取消 ⛔

四、数据表结构

共 7 张表。格式说明：**PK** = 主键，**FK** = 外键，**UQ** = 唯一约束。

总览 — 表关系图



1. Users — 用户表

存顾客、商家、骑手三类用户，通过 `role` 字段区分。

列名	类型	约束	说明
user_id	INT	PK , IDENTITY(1,1)	用户 ID，自增
username	NVARCHAR(50)	NOT NULL, UQ	用户名，不可重复
password_hash	NVARCHAR(255)	NOT NULL	密码哈希（scrypt 加密存储）
real_name	NVARCHAR(50)	NOT NULL	真实姓名

列名	类型	约束	说明
phone	NVARCHAR(20)	NOT NULL	手机号码
address	NVARCHAR(200)	可空	地址（顾客的默认送餐地址）
role	NVARCHAR(20)	NOT NULL，默认 'customer'	角色：customer（顾客） / merchant（商家） / rider（骑手）
created_at	DATETIME2	NOT NULL，默认 GETDATE()	注册时间

2. Restaurants — 商家表

每个商家由一个 merchant 角色的用户拥有。

列名	类型	约束	说明
restaurant_id	INT	PK, IDENTITY(1,1)	店铺 ID，自增
owner_id	INT	FK → Users.user_id, ON DELETE CASCADE	店主（必须是 role=merchant 的用户）
name	NVARCHAR(100)	NOT NULL	店铺名称
address	NVARCHAR(200)	NOT NULL	店铺地址
phone	NVARCHAR(20)	NOT NULL	店铺联系电话
description	NVARCHAR(500)	可空	店铺简介
status	NVARCHAR(20)	NOT NULL，默认 'open'	营业状态：open（营业中） / closed（歇业）
created_at	DATETIME2	NOT NULL，默认 GETDATE()	创建时间

3. MenuCategories — 菜品分类表

每个商家自定义自己的菜品分类（如：招牌热菜、凉菜、主食、饮品）。

列名	类型	约束	说明
category_id	INT	PK, IDENTITY(1,1)	分类 ID，自增
restaurant_id	INT	FK → Restaurants.restaurant_id, ON DELETE CASCADE	所属店铺
name	NVARCHAR(50)	NOT NULL	分类名称
sort_order	INT	NOT NULL，默认 0	排序号，越小越靠前

4. MenuItems — 菜品表

每道菜属于某个商家下的某个分类。

列名	类型	约束	说明
item_id	INT	PK , IDENTITY(1,1)	菜品 ID, 自增
restaurant_id	INT	FK → Restaurants.restaurant_id	所属店铺
category_id	INT	FK → MenuCategories.category_id, ON DELETE CASCADE	所属分类
name	NVARCHAR(100)	NOT NULL	菜品名称
description	NVARCHAR(500)	可空	菜品描述
price	DECIMAL(10,2)	NOT NULL, CHECK ≥ 0	单价
status	NVARCHAR(20)	NOT NULL, 默认 'available'	available (上架) / unavailable (下架)

5. Orders — 订单表

核心业务表。记录了顾客下单、商家接单、骑手配送的完整链路。

列名	类型	约束	说明
order_id	INT	PK , IDENTITY(1,1)	订单 ID, 自增
customer_id	INT	FK → Users.user_id	下单顾客
restaurant_id	INT	FK → Restaurants.restaurant_id	接单商家
rider_id	INT	FK → Users.user_id, 可空	配送骑手 (配送前为 NULL, 骑手接单后填入)
delivery_address	NVARCHAR(200)	NOT NULL	配送地址
status	NVARCHAR(20)	NOT NULL, CHECK 约束 8 种状态, 默认 'pending'	订单当前状态
total_amount	DECIMAL(10,2)	NOT NULL, CHECK ≥ 0, 默认 0	订单总金额 = 菜品合计 + 配送费
delivery_fee	DECIMAL(10,2)	NOT NULL, CHECK ≥ 0, 默认 5.00	配送费
note	NVARCHAR(500)	可空	订单备注 (少放辣、不要糖等)
pickup_time	DATETIME2	可空	骑手取餐时间

列名	类型	约束	说明
delivery_time	DATETIME2	可空	骑手送达时间
created_at	DATETIME2	NOT NULL, 默认 GETDATE()	下单时间

6. OrderItems — 订单明细表

一个订单可以包含多道菜品，每道菜单独一条记录。`unit_price` 是下单时的快照价格。

列名	类型	约束	说明
order_item_id	INT	PK , IDENTITY(1,1)	明细 ID, 自增
order_id	INT	FK → Orders.order_id, ON DELETE CASCADE	所属订单
item_id	INT	FK → MenuItems.item_id	菜品 ID
quantity	INT	NOT NULL, CHECK > 0	数量
unit_price	DECIMAL(10,2)	NOT NULL, CHECK ≥ 0	下单时的菜品单价（快照，不随后续菜品改价而变化）

7. Reviews — 评价表

一个订单只能评价一次（UQ on order_id），评分 1-5 分。

列名	类型	约束	说明
review_id	INT	PK , IDENTITY(1,1)	评价 ID, 自增
order_id	INT	FK → Orders.order_id, ON DELETE CASCADE, UQ	评价的订单（一个订单只能评价一次）
customer_id	INT	FK → Users.user_id	评价人（下单顾客）
rating	SMALLINT	NOT NULL, CHECK 1~5	评分，1-5 分
comment	NVARCHAR(500)	可空	文字评价
created_at	DATETIME2	NOT NULL, 默认 GETDATE()	评价时间

五、索引设计策略

5.1 设计原则

原则	说明
外键必建索引	所有 FK 列均建有非聚集索引，保证 JOIN 和参照完整性检查的效率
筛选列建索引	role、status 等高频出现在 WHERE 子句的列建有索引
OLTP 优先	以支持高频短事务为主；复合索引暂未引入，避免写入放大
基数不歧视	低基数列（如 status 仅 2~8 个值）依然建索引 —— 因为查询总是带 WHERE status = '...' 条件

5.2 索引清单

表	索引名	索引列	类型	对应的典型查询
Users	IX_Users_Role	role	非聚集	按角色筛选用户
Restaurants	IX_Restaurants_Status	status	非聚集	顾客浏览营业中商家：WHERE status = 'open'
Restaurants	IX_Restaurants_Owner	owner_id	非聚集	商家查找自己的店铺：WHERE owner_id = ?
MenuCategories	IX_Categories_Restaurant	restaurant_id	非聚集	加载商家菜单分类：WHERE restaurant_id = ?
MenuItems	IX_Items_Restaurant	restaurant_id	非聚集	加载商家菜品列表：WHERE restaurant_id = ?
MenuItems	IX_Items_Category	category_id	非聚集	按分类加载菜品：WHERE category_id = ?
MenuItems	IX_Items_Status	status	非聚集	仅展示上架菜品：WHERE status = 'available'
Orders	IX_Orders_Customer	customer_id	非聚集	顾客查看"我的订单"：WHERE customer_id = ?

表	索引名	索引列	类型	对应的典型查询
Orders	IX_Orders_Restaurant	restaurant_id	非聚集	商家查看"本店订单": WHERE restaurant_id = ?
Orders	IX_Orders_Rider	rider_id	非聚集	骑手查看"我的配送": WHERE rider_id = ?
Orders	IX_Orders_Status	status	非聚集	按状态筛选; 骑手查看待接订单: WHERE status = 'ready'
OrderItems	IX_OrderItems_Order	order_id	非聚集	加载订单明细: WHERE order_id = ?
Reviews	IX_Reviews_Order	order_id	非聚集	按订单查评价: WHERE order_id = ? (也由 UNIQUE 约束保障)

共计 13 个索引。主键 (*_id) 均由 PRIMARY KEY 自动创建聚集索引, 不在表中重复列出。

5.3 主要查询路径与索引命中

查询场景	对应路由	索引路径	备注
顾客浏览商家列表	restaurant/list_restaurants()	IX_Restaurants_Status	WHERE status='open', 全表扫描过滤关键字
加载商家菜单	restaurant/detail(id)	IX_Categories_Restaurant + IX_Items_Category + IX_Items_Status	两级加载: 分类 → 菜品 (仅 available)
顾客查看订单	order/my_orders() (customer)	IX_Orders_Customer + IX_Orders_Status	按 customer_id 过滤, 可选按状态分组
商家查看订单	order/my_orders() (merchant)	IX_Orders_Restaurant + IX_Orders_Status	先查 Restaurant → 再查 Orders
骑手查看待接订单	delivery/available_orders()	IX_Orders_Status + IX_Orders_Rider	WHERE status='ready' AND rider_id IS NULL
骑手查看配送记录	delivery/my_deliveries()	IX_Orders_Rider + IX_Orders_Status	WHERE rider_id=? AND status IN (...)

查询场景	对应路由	索引路径	备注
订单详情 + 明细	order/detail(id)	PK(Orders) + IX_OrderItems_Order	主键点查 + FK 索引 JOIN
提交评价	order/review(id)	PK(Orders) + IX_Reviews_Order	先查订单 → 再查/写评价

六、约束设计

6.1 约束总览

CHECK role ∈ {cust, merch, rider} status ∈ 合法枚举（每表独立定义） price / amount / fee ≥ 0 quantity > 0 rating BETWEEN 1 AND 5		
FK ON DELETE CASCADE ON DELETE 默认	UNIQUE username order_id	DEFAULT role = 'customer' status = 初始值 *_at = GETDATE() delivery_fee = 5

6.2 CHECK 约束

所有 CHECK 约束在数据库层定义，作为最后防线；应用层（orderState 状态机）也做校验，但 DB 层确保非法数据根本无法写入。

表	列	CHECK 表达式	目的
Users	role	IN ('customer', 'merchant', 'rider')	角色仅限三种合法值
Restaurants	status	IN ('open', 'closed')	营业状态二值
MenuItems	price	>= 0	防止负价格
MenuItems	status	IN ('available', 'unavailable')	菜品上下架二态
Orders	status	IN ('pending', 'confirmed', 'preparing', 'ready', 'assigned', 'picked_up', 'delivered', 'cancelled')	限 8 种合法状态

表	列	CHECK 表达式	目的
Orders	total_amount	<code>>= 0</code>	防止负金额
Orders	delivery_fee	<code>>= 0</code>	防止负配送费
OrderItems	quantity	<code>> 0</code>	数量必须为正整数（不允许 0）
OrderItems	unit_price	<code>>= 0</code>	下单快照单价非负
Reviews	rating	<code>BETWEEN 1 AND 5</code>	评分 1~5 星

6.3 FOREIGN KEY 约束与级联策略

子表	FK 列	引用父表	ON DELETE	决策理由
Restaurants	owner_id	Users	CASCADE	用户注销 → 店铺随之删除，避免孤儿店铺
MenuCategories	restaurant_id	Restaurants	CASCADE	店铺删除 → 分类随之删除
MenuItems	restaurant_id	Restaurants	NO ACTION	保留菜品数据，删除店铺前需手动处理
MenuItems	category_id	MenuCategories	CASCADE	分类删除 → 旗下菜品随之删除
Orders	customer_id	Users	NO ACTION	必须保留历史订单 ，不能因用户注销而丢失
Orders	restaurant_id	Restaurants	NO ACTION	必须保留历史订单 ，不能因店铺关闭而丢失
Orders	rider_id	Users	NO ACTION	保留配送记录，骑手信息可追溯
OrderItems	order_id	Orders	CASCADE	订单删除 → 明细随之删除
OrderItems	item_id	MenuItems	NO ACTION	保留历史明细的快照数据，即使菜品已删除
Reviews	order_id	Orders	CASCADE	订单删除 → 评价随之删除
Reviews	customer_id	Users	NO ACTION	保留评价数据

核心原则：以订单为中心的数据（Orders、OrderItems、Reviews）之间有 CASCADE 关系；但它们与用户/商家/菜品之间使用 NO ACTION，确保历史数据可追溯。

6.4 UNIQUE 约束

表	列	约束说明
Users	username	用户名全局唯一（由 NOT NULL UNIQUE 定义）
Reviews	order_id	一个订单只能评价一次（由 NOT NULL UNIQUE 定义，同时自动建索引）

6.5 DEFAULT 值

表	列	默认值	说明
Users	role	'customer'	注册默认顾客身份
Users	created_at	GETDATE()	自动记录注册时间
Restaurants	status	'open'	新建店铺默认营业
Restaurants	created_at	GETDATE()	自动记录创建时间
MenuCategories	sort_order	0	新建分类排序号从 0 开始
MenuItems	status	'available'	新建菜品默认上架
Orders	status	'pending'	下单即进入待处理状态
Orders	total_amount	0	先占位，由下单逻辑计算实际金额后覆盖
Orders	delivery_fee	5.00	默认配送费 ¥5，下单时从应用配置读取
Orders	created_at	GETDATE()	自动记录下单时间
Reviews	created_at	GETDATE()	自动记录评价时间

6.6 设计决策总结

决策	选择	理由
状态字段类型	NVARCHAR(20) 而非 TINYINT	可读性优先，方便在数据库中直接查看和调试
级联删除范围	仅"强依赖"链 CASCADE	防止误删历史数据；用户/商家/菜品删除不影响已有订单
菜品价格	OrderItems.unit_price 做快照	菜单调价不影响历史订单金额
配送费	DB 默认 ¥5 + 应用层可覆盖	灵活性和一致性兼顾
状态校验	DB CHECK + 应用层状态机	双重保障；DB 是最后防线，应用层提供友好错误提示
索引策略	全覆盖 FK + 常用筛选列	← 见第五节

对应 SQL 文件： `sql/01_create_tables.sql`（建表）、`sql/02_insert_sample.sql`（示例数据）。
